

가정	성능지표	α_i / β_i change	알고리즘	알고리즘 분류
$r_i = 0$ $s_{ij} = 0$ C, max는 상수 $= \sum(p, j)$ 고의적 유희시간 발생 x r, d: 고정	Total completion time	$1 C(F \text{ or } L)$	SPT 규칙	건설적 알고리즘 (최적화 알고리즘) Polynomial
	Weighted mean flow time $((\sum_{i=1}^n \alpha_i f_i) / n)$	$1 \sum \frac{\alpha_i f_i}{n}$ n없어도됨	WSPT 규칙	
	Maximum lateness	$1 L_{\max} \text{ or } T_{\max}$	EDD 규칙	
	Number of tardy jobs	$1 n_T$	Moore 알고리즘	
	Maximum penalty $(\max_{i=1}^n \{f_i(C_i)\})$	$1 \max_{i=1}^n \{f_i(C_i)\}$	Lawler 알고리즘	
$r=0$ 이면, r 이면 = 최적	Any	1Any	동적계획법	암묵적 열거법 (최적화 알고리즘)
	Total tardiness	$1 \sum_{i=1}^n T_i$	문제에 특화된 분지 한정법	
	Performance measure of total or average types	$1 \text{Total of Average}$	LP 완화 기반 분지 한정법	
	Any	1Any	타부탐색 시뮬레이티드 어닐링 유전자 알고리즘	

① SPT : $\bar{F}$ 최소화. ($C, L, \bar{F}, \bar{L}$ 도 최소화)

→ 가용시간(C_p)가 작을수록 큰 k 까지 스케줄 (순서: k)

$$\bar{F} = \frac{1}{n} \sum_{k=1}^n (n-k+1) p_{ik}(k) \quad F = W + p, \quad C = r + F, \quad L = C - d$$

② WSPT: 가중체취시간. (α : 비용 or 시급성). $\frac{p_i}{d_i}$ 가 작은 순서대로 스케줄. ($p \downarrow, d \uparrow$ 일수록 많음)

③ EDD: 가장 늦은 작업의 지연시간 L_{max} . L_{max} 최작라 $\rightarrow T_{max}$ 최작라. (반대성질)
 $\hookrightarrow d(\text{작가})$ 가 빠른 순서로 스케줄. $\hookrightarrow$ 이유: $T_{max} = 0$ 이어도 L_{max} 줄일 수 있음.

④ Moore : n_T 회전바

Step 1. EDD 규칙에 따라 작업을 정렬 한 후 다음 조건을 만족하는 현재 스케줄을 찾는다. 따라서 현재 스케줄 $(J_{(1)}, J_{(2)}, \dots, J_{(n)})$ 는 $d_{(i)} \leq d_{(i+1)}, i = 1, 2, \dots, n-1$ 를 만족한다.

Step 2. 현재 스케줄에서 첫 번째 지연 작업 $J_{(k)}$ 를 찾는다. 만약 지연된 작업이 없다면 Step 4로 이동한다.

Step 3. 부분 스케줄 $(J_{(1)}, J_{(2)}, \dots, J_{(k)})$ 에서 가공시간이 가장 큰 작업을 제거하고 Step 2 로 되돌아간다. 현재 스케줄이 전 보다 하나 짧아 진다.

Step 4. 현재 스케줄의 뒤에 Step 3에서 제거된 작업을 붙여서 최적 스케줄을 얻는다.
(제거된 작업의 순서는 상관이 없다).

⑤ Lawler : 작업성행수 (prec)가 낮을수록 regular measure $f_L(C_i)$ 증가

최대값을 최소화 하기 위한 스케줄링 기법. $(|prec| \max_{i=1}^n \{f_i(c_i)\})$

$$f_i(C_i) = C_{\max}, F, F_{\max}, L, L_{\max}, T, T_{\max}, n_T$$

Step 1. 스케줄 할 수 있는 모든 작업의 총 작업완료시점 τ

그리고 스케줄의 끝에 위치할 수 있는 작업의 집합 V 를 구성한다. $(V) = 1$ 이면 종료한다. V 안에서만 고민하면 됨!!

Step 2. $f_k(\tau) = \min_{J_i \in V} \{f_i(\tau)\}$ 인 작업 J_k 를 발견하여 스케줄의 맨 뒤에 위치시킨다.
 $V = V - \{J_k\}$ 로 변경하고 Step 1으로 돌아간다.

$\frac{1}{2} \times 10 \times 10 = 50$

중역시작 (forward DP: 현재 위치 현재 상태) (transition: $s_j \rightarrow x_j \rightarrow s_{j-1}$)

$$f(s_j) = \min_{\substack{\text{의사결정 } a \in A(s_j) \\ \text{모든 의사결정}}} \left\{ \underbrace{\text{의사결정에 따른 보상}}_{\text{immediate reward}} + \gamma \underbrace{\text{다음 단계에서의 최적 가치}}_{\text{max expected future reward}} \right\}$$

- 이제 일정계획 문제를 동적 계획법으로 풀어보기 위해 단일기계에서의 목적함수 Z 를 다음과 같이 각 작업 i 의 완료시점 C_i 의 함수 $g(C_i)$ 의 합이라고 생각해 보자.

Explicit $n!$ 개 곱
Implicit $n2^n \rightarrow O(n^2 2^n)$

$Z = \sum_{i=1}^n g(C_i) \rightarrow \min Z$

작업의 수 $n!$ 이 증가함
2개 인공의 상태 (부분집합)에 대해서

- $g(C_i)$ 의 형태에 따라 다음과 같이 다양한 목적함수를 구성할 수 있다.

- $g(C_i)$ 가 $\max\{0, C_i - d_i\}$ 이면 목적 함수 Z는 **순수지연시간**의 합이 된다.
- $g(C_i) = w_i \delta[\max\{0, C_i - d_i\}]$ 이고 δ 를 Indicator함수, w_i 를 작업 i 의 가중치라고 하면 목적 함수 Z는 가중 순수지연 작업수가 된다.

- n 개 작업의 일정계획을 위한 단계, 상태, 의사결정, 보상을 다음과 같이 정의

단계	작업의 부분집합 크기로 정의한다. 작업이 총 n 개 있으므로 부분집합 크기는 1부터 n 까지 존재한다. 즉 단계는 1부터 n 까지 존재한다. 5개를 해해야 하는 작업
상태	작업의 부분집합으로 정의한다. $n = 3$ 이고 단계가 2이면 가능한 상태는 $\{1, 2\}, \{1, 3\}, \{2, 3\}$ 이다. 3개를 2개를 5개를 해해야 함
의사결정	스케줄의 마지막 위치까지 작업을 결정하는 것으로 정의한다. $n = 3$ 이고 단계가 2, 상태가 $\{2, 3\}$ 이면 마지막 작업은 작업 2 또는 작업 3이 될 수 있다.
보상	의사결정에 따른 $g(C_i)$ 값이다. $g(C_i)$ 가 $\{0, C_i - d_i\}$ 이라고 생각해 보자. $n = 3$ 이고 단계가 2, 상태가 $\{2, 3\}$, 의사결정인 마지막 작업을 작업 2라고 하면 보상은 $g(C_2)$ 즉 작업 2의 순수치연산시간이 된다.

- **단계 1:** 가능한 상태 S_1 은 $\{1\}, \{2\}, \dots, \{n\}$ 이 된다. 각 상태마다 재귀식은 다음과 같다.

$$f_1(S_1) = \min_{i \in S_1} [g(C_i) + f_0(i)] = \min_{i \in S_1} [g(C_i)]$$

- 단계 $j(2 \leq j \leq n)$: 가능한 상태 S_j 은 총 작업 n 개 중에 j 개 작업으로 구성된 부분집합들이 된다. 각 상태마다 재귀 식은 다음과 같다.

$$f_j(S_j) = \min_{i \in S_j} [g(C_i) + f_{j-1}(S_j - \{i\})]$$

- 목적함수에 대한 재귀식을 예를 들어 정의해보면 다음과 같다. $n = 3$ 이고 단계가 2, 상태가 {2, 3}일 때 최적의 목적함수 값은 다음과 같이 표현된다. Job 1 2

가장 마지막에 선택 $f_2(\{2, 3\}) = \min_{i \in \{2, 3\}} [g(C_i) + f_1(\{2, 3\} - \{i\})]$ $\rightarrow p_2 + p_3 - d_i$

Example 3 S_3 $\uparrow$ $\{1, 2, 3\}$ $\{1, 2, 4\}$ $\{1, 3, 4\}$ $\{2, 3, 4\}$ Example 4 S_4 $\{1, 2, 3, 4\}$
 $i \in S_i$ $\{1, 2, 3\}$ $\{1, 2, 4\}$ $\{1, 3, 4\}$ $\{2, 3, 4\}$ $i \in S_i$ 1 2 3 4

$g(c_i)$

28-9	28-7	28-11	28-13
=19	=21	=17	=15

$p_1 + p_2 + p_3 - d_1$

$f(n)$: 크기가 n 인 문제를 푸는데 필요한 최대 기본연산수.
 $f(n) = O(g(n))$ whenever $\frac{f(n)}{g(n)} \rightarrow c$ for constant c as $n \rightarrow \infty$

$O(n^m)$: polynomial (단계수) $\times$ (단계마다 처리할 양)
 $O(n!), O(x^n)$ ($x \geq 2$) : exponential
 시간복잡도 : 입력 크기 n 이 커졌을 때 알고리즘이 수행해야 하는 연산수.

Linear Search $O(n)$. Binary Search $O(\log_2 n)$. Bubble sort $O(n^2)$
 Merge Sort $O(n \log_2 n)$. Quick sort $O(n \log_2 n) \sim O(n^2)$. 피보나치 $O(2^n)$
 ① Shortest Path (Dijkstra : $O(n^2)$, Floyd : $O(n^3)$). LP : $O(2^n \cdot m \cdot n)$ 최단경로
 EDD, SPT, Moore $O(n \log_2 n)$ 정렬기법에 따라 달라짐. Lawler : $O(n^3)$ 후방작업 n^2
 Frac. knapsack $O(n \log_2 n)$. Min. span $O(E \log V)$
 ② NP Boolean satisfiability $O(2^n)$. Hamilton cycle $O(\frac{(n-1)!}{2})$
 TSP $O(n!)$ 최단. $O(n^2 \cdot 2^n)$ 다변형. knapsack $O(n2^n)$ 다변형. 최단
 VRP $O(2^n)$ 의사변형. $O(n!m!)$ 최단

	P	NP	NP-C	NP-H	
의사변형	o	o	o	x	(해결됨)
최적화	o	x	x	x	(해 찾기)

NP-H : NP-Complete
 NP : NP-Complete
 P : Polynomial

[성능지표]

← NPM → ← 시간 →

$C_i = r_i + w_i + p_i$, $F_i = C_i - r_i = w_i + p_i$, $L_i = C_i - d_i$ (음수 가능), $T_i = \max\{L_i, 0\}$
 C_{max} , $\bar{C}$ F_{max} , $\bar{F}$ L_{max} , $\bar{L}$ T_{max} , $\bar{T}$

• Equivalence : 성능지표 π_1 최적화 $\rightarrow \pi_2$ 최적화 $[C, F, L, eq]$ $C = F + r = L + d$
 $\hookrightarrow C_m, F_m, L_m$: not eq. $C_m \neq F_m + r_m \neq L_m + d_m$ (또는 r, d 가 같다면 eq.)
 $\hookrightarrow N, C/C_m$: eq ($N = \sqrt{N}$ 재분할) $\hookrightarrow L_m$ 최단하면 T_m 최단. (반대성립 X. L 수가 있음)
 N & C_m : 반비례. 유휴시간을 넣어도 짧은 작업을 먼저 하기에 C_m 관점에서 더 유리.

• 단일기계에서 C_{max} 는 불변. $S_{ij} \neq 0$ 이면 변함
 $r_i = 0$ 비연 $N = F/C_{max} = C/C_{max}$ 와 C : eq.

• Reg. Measure : C_i 에 대해 비교소 성능지표 ex) C_m, F_m, L_m, T_m, n_T
 $\hookrightarrow$ 스케줄 집합 A가 집합 B에 의해서 dominate $\rightarrow A$ 는 더 이상 고려 X.
 ex) 단일기계 : 비연 스케줄이 지연 스케줄 dominate.

① Reg. 최소화 $\rightarrow$ 균형성만 고려 $\rightarrow$ 활성을 제외한 균형성 스케줄 집합은
 ② " $\rightarrow$ 활성만 고려 $\rightarrow$ 활성 스케줄 집합에 의해 Dominate
 ③ " $\rightarrow$ 비연만 고려 X

• Efficiency : 스케줄 S' 가 efficiency 하다면 모든 π 에 대해서 동시에 더 나을 수 X.

[기계한정]

• 단일(1) : $njobs \rightarrow \square$

• 병렬 $\begin{cases} \text{동일병렬(PM)} & p_{im1} = p_{im2} = p_i \\ \text{균일(QM)} & \text{기계마다 가공속도 } r_m \text{ 존재 } p_{im} = p_i \times r_m \\ \text{독립(RM)} & r_m \text{ 정의 안됨} \end{cases}$

• 흐름착상장(FM) $njobs \rightarrow \square \dots \square$

• 유연흐름착상장(FFC) $njobs \rightarrow (\square) \rightarrow (\square) \rightarrow \dots (\square)^{mstage}$

• 개방착상장(OM) $\square \dots \square$

• 개별착상장(JC) $(\square) \rightarrow (\square) \rightarrow \dots (\square)$

한유시 : FM, FFC, stage 3 이상 : FFC, JC

[활성스케줄]

Step 1. $t = 1, P_t = \emptyset, S_t$ 는 선행하는 오퍼레이션이 없는 모든 오퍼레이션의 집합으로 설정한다. $\rightarrow$ 가장 먼저 끝낼 수 있는 작업이 존재함

Step 2. $\Phi^* = \min_{o_k \in S_t} \{\Phi_k\}$ 를 찾는다. 그리고 Φ^* 오퍼레이션이 가공되는 기계를 M^* 이라 정의하자. 만일 M^* 이 두 개 이상이면 임의로 하나를 선택한다. $S_{t+1} \leftarrow M^*$ 찾기!

Step 3. S_t 에 있는 오퍼레이션 중에서 다음 두 조건을 만족하는 오퍼레이션 o_j 를 찾는다.

- 조건 (1): 오퍼레이션 o_j 는 기계 M^* 에서 가공된다.
- 조건 (2): $\sigma_j < \Phi^*$ $\rightarrow$ $\Phi^* < \sigma_j$ 인 이호선 선택하면 σ_k 가 더 작을 수 있음. 즉, 활성 X.
- 조건 (1)과 (2)를 만족하는 오퍼레이션은 항상 한 개 이상 존재하며 두 개 이상이면 임의로 하나를 선택한다.

Step 4. $P_{t+1} = P_t + \{o_j\}, S_{t+1} = S_t - \{o_j\} + \{o_j \text{의 후속 오퍼레이션}\}, t = t + 1$.

Step 5. 만일 $S_t \neq \emptyset$ 이면 Step 2로 되돌아가고, 그렇지 않다면 종료한다.

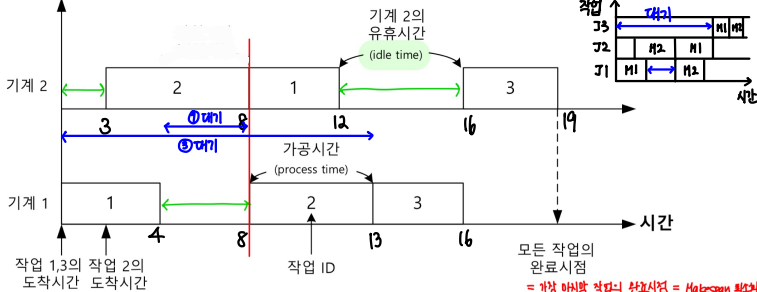
스케줄링(scheduling: 일정계획)은 한정된 자원(resource)을 다수의 작업(job)이 배타적이고 경쟁적으로 사용하고자 할 때 자원을 효율적으로 사용하는 작업들의 스케줄을 만들어 시스템에서 발생하는 비용을 최소화(또는 이익을 최대화)하는 방법론을 다루는 운영과학(operations research)의 한 분야

- 스케줄: 각 자원에서 작업들의 할당 순서(sequence)와 실행시점(timing)
- 스케줄링 문제 $\rightarrow$ 최적화 문제: 개별 작업이나 자원 입장이 아니라 모두를 포함한 시스템 차원에서 목적에 부합하도록 결정

목적식: Minimize(or Maximize) 성능지표
 제약식: Subject to 작업 및 자원에 대한 제약 조건
 결정변수: 스케줄(작업의 할당 순서와 실행시점)

• 성능지표: 스케줄이 얼마나 좋은지를 측정하는 지표로서 결정변수인 스케줄의 함수로 정의, 예: 모든 작업들의 완료시점

기계 ID 공집합에 대한 제약과 작업 집합은 표현되지 않음.



• 작업관점에서의 대기시간 • 기계관점에서의 유휴시간 (조각시간유의)

- 도착시간에 맞춰 작업을 바로 시작하는 것이 최선이 아닐 수도 있음. (과의적 유휴발생)

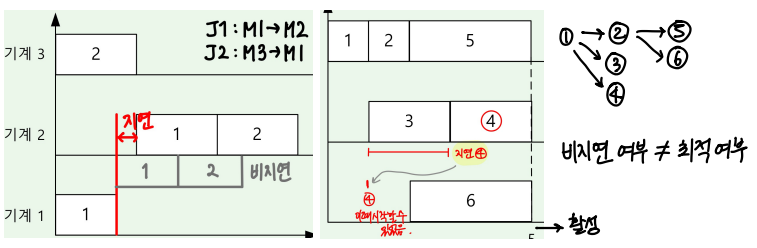
- 작업순서(Ck) & 실행시점 모두 결정변수.

[이해해답법] Step 1. 최소스케줄 생성 (해배해조 설정). Step 2. 이어서 생성 $\rightarrow$ ① Single adj. 바꾼 작업 (n-1)개 생성. ② Single pairwise: 쌍의 합서 $\frac{n(n-1)}{2}$ 생성

Step 3. 이어서 하나 선택. Step 4. 종료조건 (개선되지)

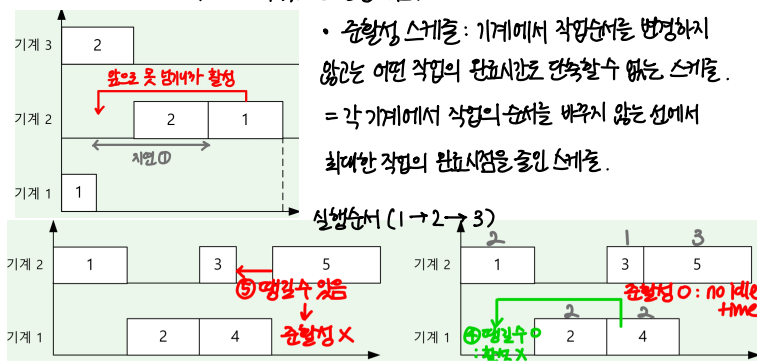
[스케줄유형]

• 비지연: 각 기계에서 대기 중인 작업이 있을 때 과의적으로 유휴시간을 발생시키지 않는 스케줄



• 활성스케줄 : 작업의 순서를 변경했을 때 다른 작업의 완료시점을 늦추지 않고는 앞으로 이동시킨 작업의 완료시점을 줄일 수 없는 스케줄을 의미 $\rightarrow$ 더 이상의 개선 X

$\hookrightarrow$ idle time에 억 들어갈 수 있으면 활성 아님.



① 균형성 확인

• 순서만 바꾸고 최대한 뺄 것인가?

균형성 $\rightarrow$ 활성확인 / 균형성 X $\rightarrow$ 최적 X

② 활성확인 : 최적의 필요조건 (not 충분)

• 빈칸에 없었으면, 다른 작업 C 끼지?

Yes : 활성 $\rightarrow$ 최적일수도 / NO : 활성 X $\rightarrow$ 최적일수도 없음.

③ 비지연, 지연 : 최적성과 무관

• 모든 비지연 스케줄은 활성. (o)

• 모든 활성 스케줄은 비지연 (X)

